

Mini Estudo 1: Baseline do Sistema Escolhido

Baseline da NVIDIA RTX 5060 Ti 16 GB para SGEMM denso (float32) via cuBLAS

Equipe:

Caio Aleixo Cunha

João Carlos Normando Nogueira

Leonardo de Souza Bitencourt Irias

O restante dos entregáveis está disponível no seguinte repositório no github:

<https://github.com/joaocnn/miniestudo1-baseline-rtx5060ti>

1. Pergunta operacional

Qual é o desempenho-base, expresso em GFLOPS e em tempo de execução (ms), da multiplicação de matrizes quadradas densas em float32 na GPU NVIDIA RTX 5060 Ti 16 GB, utilizando a biblioteca cuBLAS, para tamanhos N pertencentes ao conjunto {256, 512, 1024, 2048, 4096}?

O estudo busca caracterizar o comportamento da plataforma sob uma carga controlada e bem definida, sem o objetivo de comparar a GPU com outros dispositivos. O baseline é entendido aqui como uma medida de referência inicial, não como prova de superioridade ou inferioridade de algum sistema.

2. Sistema avaliado

O sistema avaliado é uma máquina hospedeira única equipada com uma GPU NVIDIA GeForce RTX 5060 Ti 16 GB (arquitetura Blackwell, compute capability 12.0), CPU AMD Ryzen 7 5700X de 8 núcleos e aproximadamente 32 GB de memória RAM. O sistema operacional foi o Ubuntu 24.04.2 LTS com kernel 6.17.0-20-generic. A GPU foi mantida em modo de energia padrão e não houve concorrência por unidades de execução compute (tipo C) durante a coleta. A interface gráfica permaneceu ativa, situação que é tratada explicitamente na Seção 8 (Limitações).

O cenário-base, portanto, é um único sistema, em uma única configuração, sob uma única carga, sem comparações entre plataformas. Os ajustes de versões, drivers e bibliotecas estão detalhados no Apêndice A.

3. Métrica e instrumento

A métrica principal é o tempo de execução do kernel GEMM, em milissegundos (ms). Como métrica derivada, reporta-se o throughput em GFLOPS, calculado de forma determinística a partir do tempo medido pela expressão $\text{GFLOPS} = (2 \cdot N^3) / (\text{tempo_em_segundos} \cdot 10^9)$, assumindo a contagem padrão de $2 \cdot N^3$ operações de ponto flutuante por execução de SGEMM denso.

O instrumento de medição utilizado foi o mecanismo de CUDA Events, acessado via CuPy (cp.cuda.Event). Esse instrumento marca pontos no stream de execução da GPU e mede o intervalo decorrido com resolução de microssegundos, sem incluir o overhead de lançamento de kernel pelo host. Timers do host (time.time, time.perf_counter) foram explicitamente evitados pela mesma razão.

O instrumento mede diretamente a métrica principal: o tempo decorrido na GPU entre dois eventos. A métrica derivada (GFLOPS) é uma transformação determinística do tempo medido, válida sob a contagem padrão de FLOPs para SGEMM denso quadrado.

4. Benchmark e carga de trabalho

4.1. Benchmark

Foi utilizado um microbenchmark próprio, que invoca cuBLAS diretamente por meio de `cp.matmul` (CuPy), sem qualquer framework intermediário (PyTorch ou TensorFlow). A escolha por um microbenchmark é deliberada: o objetivo é caracterizar o kernel GEMM nesta GPU, e não o comportamento de uma pilha de software de mais alto nível.

4.2. Carga

A carga consiste na multiplicação $C = A \cdot B$ de matrizes quadradas $N \times N$ em precisão float32, com $N \in \{256, 512, 1024, 2048, 4096\}$. As matrizes A e B são preenchidas com valores aleatórios uniformes em $[0, 1)$ gerados a partir de uma seed fixa (42). A alocação ocorre na memória da GPU; cópias $\text{host} \leftrightarrow \text{device}$ ficam fora da região cronometrada, de modo que a medição reflete apenas o trabalho do kernel.

4.3. Caracterização da carga

- Estrutura: matrizes 100% densas, distribuição uniforme $[0,1)$, layout contíguo (column-major do cuBLAS).
- Volume de trabalho: $2 \cdot N^3$ FLOPs por execução, variando de $3,4 \cdot 10^7$ FLOPs ($N=256$) a $1,37 \cdot 10^{11}$ FLOPs ($N=4096$).
- Footprint de memória máximo: 192 MB para $N=4096$ (três matrizes de $4096^2 \cdot 4$ bytes), bem abaixo dos 16 GB disponíveis na GPU.
- Tipo de carga: compute-bound para N grandes; memory/launch-bound para N pequenos (esta hipótese é discutida na Seção 7).

5. Protocolo de coleta

- Foram realizadas 30 medições por configuração de N (5 tamanhos $\times$ 30 = 150 medições no total).
- Antes de cada bloco de N, foram executados 5 warm-up runs descartados, com o objetivo de estabilizar clocks e cache antes da medição.
- As 30 repetições de cada N foram executadas sequencialmente, sem paralelismo entre processos.
- Cada repetição foi gravada individualmente no CSV no momento da medição, preservando os dados brutos sem agregação prévia.
- A temperatura da GPU foi registrada a cada 5 iterações via `nvidia-smi`, e uma flag automática (`delta_temp_alto`) foi gravada quando a variação intra-bloco excedeu 5 °C.
- Mantidos constantes durante toda a coleta: driver NVIDIA 570.211.01, CUDA Toolkit 12.9, cuBLAS 120901, kernel do SO, seed das matrizes (42) e ordem sequencial de execução.

O dado bruto registrado contém as colunas: `run`, `cenário`, `N`, `tempo_ms`, `gflops`, `gpu_temp_c`, `timestamp` e `observação`. Os metadados completos do ambiente (saída do `nvidia-smi`, versões de Python, CuPy, NumPy, pandas, scipy, modelo de CPU e RAM) foram persistidos em arquivo JSON separado e estão sumarizados no Apêndice A.

6. Resultados

6.1. Resumo estatístico do tempo de execução

A Tabela 1 apresenta o resumo estatístico do tempo de execução (ms) por tamanho de matriz, com 30 repetições por configuração. O intervalo de confiança de 95% para a média foi calculado com a distribuição t de Student, com 29 graus de liberdade.

N	n	Média (ms)	Mediana (ms)	Mín (ms)	Máx (ms)	DP (ms)	IC95 inf	IC95 sup	CV%
256	30	0,038	0,039	0,010	0,092	0,016	0,032	0,043	42%
512	30	0,053	0,057	0,028	0,077	0,012	0,048	0,057	23%
1024	30	0,179	0,182	0,156	0,208	0,015	0,174	0,185	8%
2048	30	1,087	1,087	1,060	1,178	0,022	1,079	1,095	2%
4096	30	8,586	8,453	8,022	9,677	0,461	8,414	8,759	5%

Tabela 1 — Resumo estatístico do tempo (ms) por tamanho de matriz N (n=30 cada).

6.2. Distribuição dos tempos por tamanho de matriz

A Figura 1 mostra a distribuição dos tempos de execução por N em escala logarítmica. Observa-se variabilidade relativa elevada para N=256 e N=512, com outliers visíveis tanto abaixo quanto acima da caixa, e distribuições compactas para N=1024 e N=2048. Para N=4096, a caixa é mais ampla do que em N=2048, refletindo o desvio padrão de 0,461 ms reportado na Tabela 1.

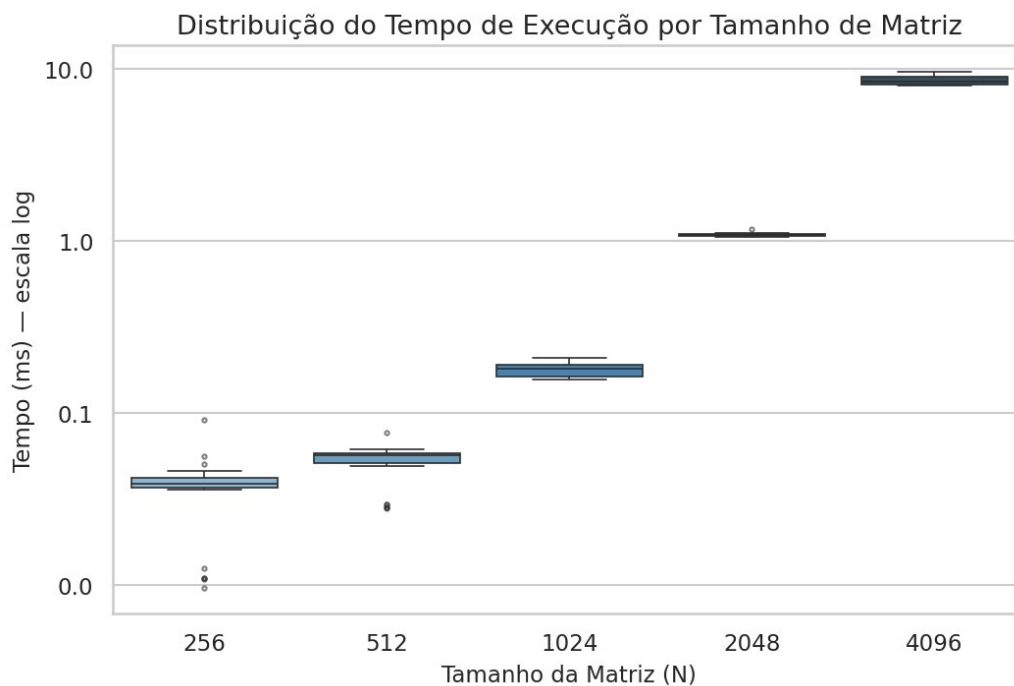


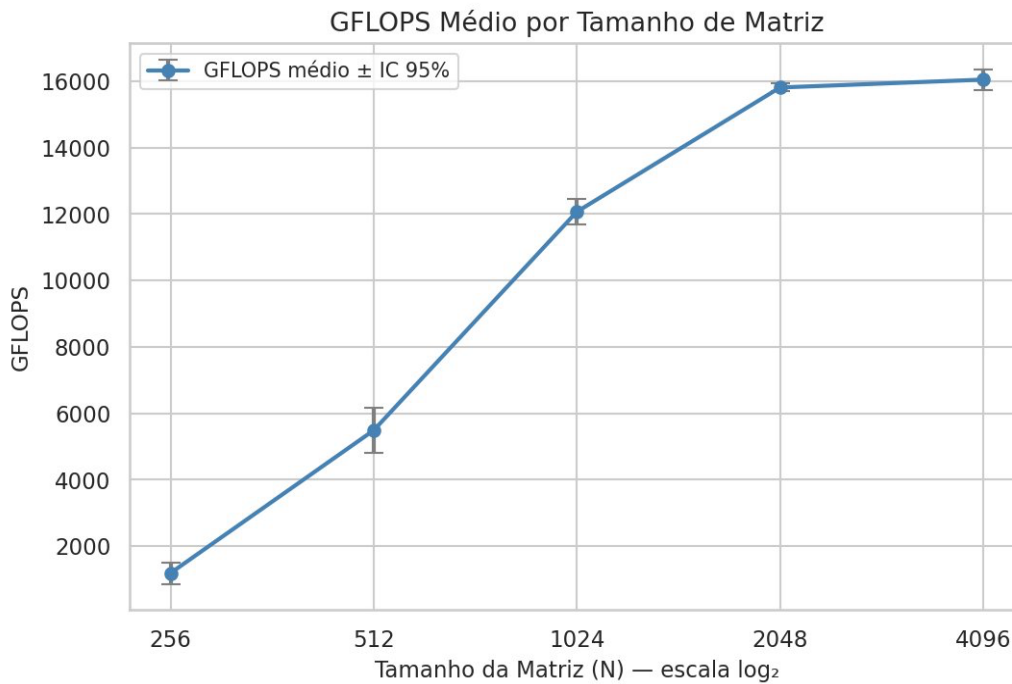
Figura 1 — Distribuição do tempo de execução (ms) por tamanho de matriz N, em escala log.

6.3. Throughput em GFLOPS

A Tabela 2 apresenta o GFLOPS médio por N, calculado a partir dos tempos médios da Tabela 1. A Figura 2 visualiza essa progressão, com barras de erro representando o IC 95%.

N	GFLOPS médio (aprox.)
256	~1.400
512	~5.500
1024	~12.100
2048	~15.800
4096	~16.100

Tabela 2 — GFLOPS médio por tamanho de matriz N.

Figura 2 — GFLOPS médio por tamanho de matriz N, com barras de erro de IC 95% (eixo X em $\log_2$).

A curva apresenta a forma clássica de saturação: crescimento aproximadamente linear no eixo log entre $N=256$ e $N=1024$, redução do ganho marginal entre $N=1024$ e $N=2048$, e platô entre $N=2048$ e $N=4096$. O ponto em que a GPU passa a operar próximo de seu pico sustentável neste cenário situa-se em $N=2048$.

6.4. Verificação da ameaça térmica

A Figura 3 mostra a temperatura da GPU ao longo das 150 medições, com separadores entre os blocos de N. A temperatura permaneceu próxima do estado de repouso para $N \leq 2048$ (entre 39 °C e 42 °C). No bloco $N=4096$, ocorreu um salto abrupto de 49 °C para 58 °C entre as iterações 10 e 11, sinalizado automaticamente como `delta_temp_alto` no CSV; a temperatura estabilizou em torno de 57–58 °C pelo restante do bloco.

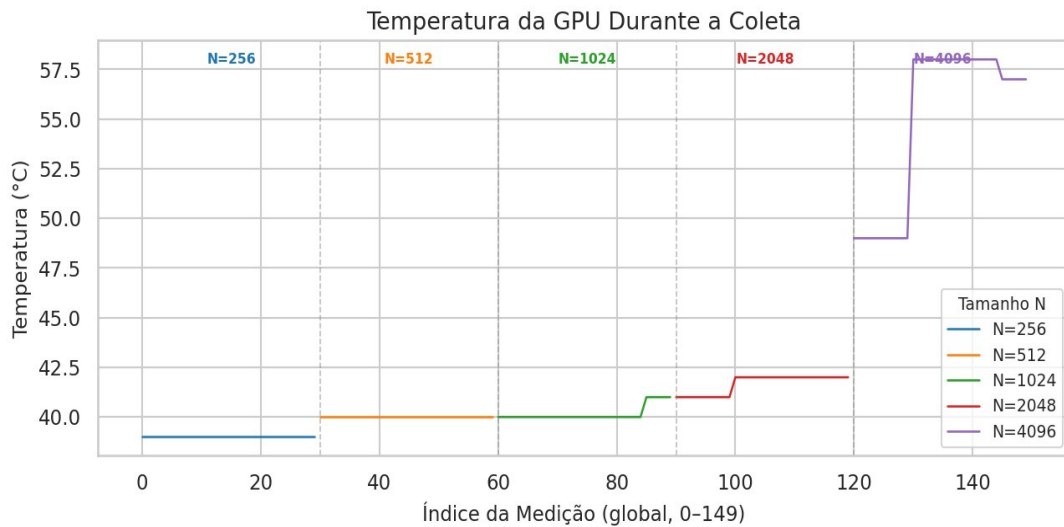


Figura 3 — Temperatura da GPU (°C) ao longo das 150 medições, com separadores visuais entre os blocos de N.

Esse comportamento confirma a ameaça à validade identificada no planejamento (variabilidade de clock dinâmico associada ao aquecimento) e contribui para o coeficiente de variação maior em N=4096 (5%) frente ao observado em N=2048 (2%).

6.5. Outliers identificados

A detecção de outliers pelo critério IQR ($k=1,5$) identificou 15 medições anômalas. A Tabela 3 sumariza a distribuição por N. O conjunto completo está no Apêndice B.

N	Outliers	Padrão observado
256	8	5 baixos periódicos (runs 6, 11, 16, 21, 26, ~0,010–0,012 ms) + 3 altos (runs 1, 13, 29)
512	6	5 baixos periódicos (runs 6, 11, 16, 21, 26, ~0,028–0,029 ms) + 1 alto (run 1, 0,077 ms)
1024	0	—
2048	1	Run 21 com 1,178 ms (outlier alto leve, compatível com preempção do scheduler do SO)
4096	0	—

Tabela 3 — Outliers identificados pelo critério IQR ($k=1,5$).

Os outliers não foram excluídos da análise estatística reportada na Tabela 1; eles são reportados explicitamente para que o leitor possa avaliar seu peso. A interpretação do padrão periódico em $N \leq 512$ é desenvolvida na Seção 7.

7. Interpretação

7.1. Padrão de saturação e ponto de operação eficiente

A combinação da Tabela 2 e da Figura 2 mostra o padrão clássico de subutilização para matrizes pequenas e saturação para matrizes grandes. O ponto de inflexão da curva está entre N=1024 (12.100 GFLOPS, aproximadamente 75% do platô) e N=2048 (15.800 GFLOPS, aproximadamente 98% do platô). O ganho marginal entre N=2048 e N=4096 é de apenas 1,9% (16.100 vs. 15.800), indicando que o pico sustentável de SGEMM denso float32 nesta GPU, neste ambiente e com este backend é da ordem de 15.800–16.100 GFLOPS.

Para $N \leq 512$, o throughput observado fica muito abaixo do pico (1.400 GFLOPS em $N=256$, 5.500 GFLOPS em $N=512$). Esse resultado é consistente com a hipótese de que, para matrizes pequenas, o tempo de execução é dominado por overheads (lançamento, sincronização, ocupação parcial dos SMs) e não pelo cálculo aritmético em si, deixando a GPU subutilizada.

7.2. Outliers periódicos em $N=256$ e $N=512$

O padrão de outliers baixos a cada 5 iterações em $N=256$ e $N=512$ (runs 6, 11, 16, 21, 26) é sistemático, não aleatório. A periodicidade exata e a magnitude (tempos de aproximadamente 1/4 a 1/2 da mediana correspondente) sugerem efeito de cache da GPU: as matrizes nesses tamanhos ocupam aproximadamente 768 KB ($N=256$) e 3 MB ($N=512$), volumes compatíveis com permanência total ou parcial no cache L2 entre execuções consecutivas em determinadas iterações.

7.3. Throttling térmico em $N=4096$

O salto térmico de 9 °C documentado na Figura 3 explica o coeficiente de variação maior em $N=4096$ ($CV = 5\%$) frente a $N=2048$ ($CV = 2\%$). O comportamento observado é consistente com uma redução dinâmica de clocks (GPU Boost) após o aquecimento, separando o bloco em duas regiões com distribuições levemente distintas: as iterações pré-salto, em temperatura mais baixa, e as iterações pós-salto, em regime térmico estabilizado em ~57–58 °C.

7.4. Configuração mais confiável

Considerando o conjunto da Tabela 1, da Figura 1 e da Figura 3, $N=2048$ é a configuração com a melhor qualidade de medição: CV de 2%, único outlier leve, distribuição simétrica (média e mediana coincidem em 1,087 ms) e ausência de anomalias térmicas. Para fins de referência futura, é o ponto mais sólido do baseline desta GPU sob esta carga.

8. Limitações

As conclusões deste estudo são restritas ao cenário medido. Em particular:

- Os resultados são válidos apenas para precisão float32, matrizes quadradas, densas, geradas com seed 42, executadas via cuBLAS 120901 sob CUDA 12.9 e driver NVIDIA 570.211.01, no ambiente de hardware/software descrito no Apêndice A.
- A coleta foi realizada com a interface gráfica ativa (Xorg + GNOME consumindo aproximadamente 503 MiB de memória da GPU em processos do tipo G). Esses processos não disputam unidades de execução compute (tipo C), mas ocupam largura de banda de memória e potência, o que pode afetar marginalmente os tempos medidos.
- A bimodalidade observada em $N=256$ e $N=512$ indica que a média não é o melhor estimador para esses tamanhos. A mediana e os IC 95% reportados devem ser interpretados com essa ressalva.
- O salto térmico em $N=4096$ reduz a comparabilidade direta deste ponto com os demais N , já que mistura dois regimes de clock no mesmo bloco.
- Não é possível afirmar, com base neste estudo, que a RTX 5060 Ti seja melhor ou pior que qualquer outro dispositivo. O baseline caracteriza apenas esta plataforma sob esta carga.
- Não é possível extrapolar os resultados para outras precisões (fp16, bf16, fp64), para GEMM batched, esparsos ou de matrizes não-quadradas, nem para outras versões de driver, CUDA ou cuBLAS.
- Não é possível extrapolar os resultados para workloads aplicados (por exemplo, treino ou inferência de redes neurais, simulações científicas etc.); o estudo cobre apenas o kernel GEMM isolado.

- O comportamento sob carga térmica sustentada por períodos longos não foi caracterizado: cada bloco terminou em poucos segundos a poucas dezenas de segundos.

9. Conclusão

O baseline coletado responde diretamente à pergunta operacional da Seção 1. Para a multiplicação de matrizes quadradas densas em float32 via cuBLAS na RTX 5060 Ti 16 GB, no ambiente descrito, observou-se: tempo médio de 0,038 ms (N=256), 0,053 ms (N=512), 0,179 ms (N=1024), 1,087 ms (N=2048) e 8,586 ms (N=4096); throughput médio de aproximadamente 1.400, 5.500, 12.100, 15.800 e 16.100 GFLOPS, respectivamente, ao longo da mesma sequência de N.

O estudo permite afirmar, dentro do escopo medido, que (i) a GPU satura em torno de N=2048 com pico sustentável da ordem de 15.800–16.100 GFLOPS; (ii) para $N \leq 512$ a GPU está subutilizada e a métrica de throughput é dominada por overheads; (iii) N=2048 é a configuração com a medição mais estável; e (iv) o aquecimento observado em N=4096 é compatível com redução dinâmica de clocks e contribui para a maior variabilidade nesse tamanho.

O estudo não permite afirmar superioridade ou inferioridade da RTX 5060 Ti frente a outras GPUs, nem extrapolar os resultados para outras precisões, formatos de matriz, bibliotecas, drivers ou workloads. Comparações entre plataformas são objeto de etapas posteriores do projeto e não fazem parte deste baseline.

Apêndice A — Metadados da coleta

Campo	Valor
GPU	NVIDIA GeForce RTX 5060 Ti
Driver NVIDIA	570.211.01
VBIOS	98.06.39.40.A0
Memória GPU	16.311 MiB
Compute capability	12.0 (arquitetura Blackwell)
Clock na coleta	2.580 MHz
CUDA Runtime	12.9
cuBLAS	120901
CPU	AMD Ryzen 7 5700X 8-Core
RAM	≈ 32 GB
SO	Ubuntu 24.04.2 LTS
Kernel	6.17.0-20-generic
Python	3.11.15
CuPy	14.0.1
NumPy	2.4.3
pandas	3.0.2
scipy	1.16.3
Temperatura no início	40 °C
Interface gráfica	Ativa (Xorg + GNOME, ≈ 503 MiB GPU em processos tipo G)
Repetições por N	30 (mais 5 warm-ups descartados por bloco)
Seed das matrizes	42

Apêndice B — Outliers identificados (IQR, k=1,5)

N	Run	Tempo (ms)	Tipo	Hipótese
256	1	0,056	Alto leve	Variação inicial; primeira execução do bloco
256	6	≈0,011	Baixo	Acerto de cache L2 (matriz ~768 KB)
256	11	≈0,011	Baixo	Acerto de cache L2
256	13	0,051	Alto leve	Ruído de scheduler
256	16	≈0,011	Baixo	Acerto de cache L2
256	21	≈0,011	Baixo	Acerto de cache L2
256	26	≈0,011	Baixo	Acerto de cache L2

N	Run	Tempo (ms)	Tipo	Hipótese
256	29	0,092	Alto	Overhead de scheduler
512	1	0,077	Alto	Primeira execução do bloco
512	6	≈0,028	Baixo	Acerto de cache L2 (matriz ~3 MB)
512	11	≈0,029	Baixo	Acerto de cache L2
512	16	≈0,028	Baixo	Acerto de cache L2
512	21	≈0,029	Baixo	Acerto de cache L2
512	26	≈0,028	Baixo	Acerto de cache L2
2048	21	1,178	Alto leve	Possível preempção do scheduler do SO

Para $N=1024$ e $N=4096$ não foram detectados outliers pelo critério IQR.

Apêndice B — Ficha de planejamento do Mini Estudo Preenchida

Grupo:

Caio Aleixo Cunha

João Carlos Normando Nogueira

Leonardo de Souza Bitencourt Irias

Trilha:

Dispositivos Pessoais

Sistema analisado:

NVIDIA GeForce RTX 5060 Ti 16 GB

1. Qual é a pergunta operacional do Mini Estudo 1?

Qual é o desempenho-base (em GFLOPS) e o tempo de execução (em ms) da multiplicação de matrizes quadradas densas em float32 na RTX 5060 Ti 16 GB, utilizando cuBLAS, para tamanhos $N \in \{256, 512, 1024, 2048, 4096\}$?

2. Qual é o sistema ou cenário-base?

GPU NVIDIA RTX 5060 Ti 16 GB, em uma única máquina hospedeira, com compartilhamento de GPU apenas com a interface gráfica do sistema operacional, com o driver e a toolkit CUDA fixados em uma versão específica documentada.

3. Qual é a métrica principal e qual é a sua unidade?

Tempo de execução do kernel GEMM, em milissegundos (ms). Métrica derivada (reportada junto): throughput em GFLOPS, calculado como $2 \cdot N^3 / (\text{tempo_em_segundos} \cdot 10^9)$. Justificativa para usar tempo como principal: é a grandeza medida diretamente; GFLOPS é uma transformação determinística dela.

4. Qual instrumento será usado para medir?

(``cudaEventRecord`` + ``cudaEventElapsedTime``), que medem o tempo decorrido na própria GPU, com resolução de microssegundos. Não utilizamos `time.time()` ou `time.perf_counter()` do python pois eles incluem overhead de lançamento de kernel e sincronização e introduzem ruído desnecessário.

5. O instrumento mede diretamente a métrica ou apenas algo relacionado?

Diretamente. CUDA Events medem o tempo decorrido entre dois pontos do stream da GPU. A métrica derivada (GFLOPS) assume que o kernel executa exatamente $2 \cdot N^3$ operações de ponto flutuante.

6. Qual benchmark ou microbenchmark será executado?

Microbenchmark chamando diretamente ``cublasSgemv`` da biblioteca cuBLAS, sem framework intermediário (sem PyTorch, sem TensorFlow). Isso reduz camadas de abstração que poderiam introduzir variabilidade.

7. Qual carga de trabalho será aplicada?

Multiplicação $C = A \cdot B$, com:

- A, B, C matrizes quadradas $N \times N$
- Tipo: float32 (precisão simples)
- $N \in \{256, 512, 1024, 2048, 4096\}$: cinco tamanhos cobrindo desde "GPU subutilizada" até "GPU saturada"
- Valores: aleatórios uniformes em $[0, 1)$, gerados com seed fixa por reprodutibilidade
- Memória: alocada na GPU; **cópias host↔device fora da região cronometrada**

8. Como a carga será caracterizada?

Densidade: 100% (matrizes densas, sem zeros estruturais)

- Distribuição dos valores: uniforme em $[0, 1)$
- Footprint de memória por configuração: $3 \cdot N^2 \cdot 4$ bytes. 192 MB para a maior dimensão de matriz.
- Operações de ponto flutuante por execução: $2 \cdot N^3$ FLOPs

9. Quantas repetições serão realizadas?

30 repetições por configuração de N (5 tamanhos $\times$ 30 = 150 medições no total). Antes das 30 repetições medidas, executar 5 warm-up que são descartados, para estabilizar clocks da GPU (evita pegar a GPU "fria" com clocks rebaixados).

10. O que será mantido constante?

- Driver NVIDIA e versão do CUDA Toolkit
 - Versão do cuBLAS
 - Modo de energia da GPU: persistence mode ON, application clocks fixos se possível
 - Linux Ubuntu
 - Apenas interface gráfica do SO ocupando espaço da GPU
 - Sem compilação/uso pesado de CPU em paralelo
 - Mesma seed para geração das matrizes
 - Execução sequencial (não paralela) das repetições
 - Sem suspender/hibernar a máquina entre medições
-

11. O que será registrado como dado bruto?

run, cenario, N, tempo_ms, gflops, gpu_temp_c, timestamp, observacao
 1, rtx5060ti_cublas_fp32, 256, 0.142, 236.7, 48, 2025-XX-XX 14:03:21, ok
 2, rtx5060ti_cublas_fp32, 256, 0.139, 241.8, 48, 2025-XX-XX 14:03:21, ok

12. Que metadados serão registrados?

Exemplo de metadados reais registrados:

```
{
  "timestamp": "20250509_120000",
  "gpu": {
    "name": "NVIDIA GeForce RTX 5060 Ti",
    "driver_version": "570.211.01",
    "vbios_version": "98.06.39.40.A0",
    "memory_total": "16311 MiB",
    "clock_base": "[N/A]",
    "clock_boost": "3090 MHz"
  },
  "cuda": {
    "runtime_version": "12.9",
    "cublas_version": "120901"
  },
  "system": {
    "uname": "Linux AI-SERVER-5060TI 6.17.0-20-generic",
    "distro": "Distributor ID:\tUbuntu\nDescription:\t",
    "kernel": "6.17.0-20-generic"
  },
  "cpu": {
    "model": "AMD Ryzen 7 5700X 8-Core Processor"
  },
  "memory": {
    "ram_total": "32780588 kB"
  },
  "python": {
    "python": "Python 3.11.15",
    "cupy": "14.0.1",
    "numpy": "2.4.3",
    "pandas": "3.0.2",
    "scipy": "1.16.3"
  },
  "nvidia_smi_full": "Sat May  9 11:16:34 2026"
}
```

13. Qual análise mínima será feita?

- Resumo estatístico por N: média, mediana, mínimo, máximo, desvio padrão (de tempo_ms e gflops)
- Incerteza: intervalo de confiança de 95% da média, via t-Student com n=30 (gl=29)
- Visualização principal: boxplot de tempo_ms (ou GFLOPS) agrupado por N: mostra distribuição, mediana e outliers de uma vez
- Visualização secundária: GFLOPS médio em função de N, com barras de erro (IC 95%): evidencia se/quando a GPU satura
- Verificação de outliers: identificar e justificar

14. O que o baseline permitirá concluir?

- O desempenho-base da RTX 5060 Ti 16 GB para SGEMM denso via cuBLAS, na faixa de N testada, neste ambiente específico
- Como o desempenho escala com N nessa GPU
- A variabilidade (desvio padrão, IC) das medições nesta configuração

15. O que o baseline não permitirá concluir?

- Que essa GPU é melhor ou pior que qualquer outra (M5, RX 9060 XT, DGX Spark), pois ela tem hardware diferente, backend diferente, OS diferente
 - O desempenho em outras precisões (fp16, bf16, fp64, int8)
 - O desempenho em GEMM batched, esparso, ou com matrizes não-quadradas
 - O desempenho em workloads reais (LLM inference, treino, etc.)
 - O comportamento sob carga térmica sustentada (>30 min) ou em outras temperaturas ambientes
 - Comportamento em outras versões de driver, CUDA ou cuBLAS
-

16. Qual é a principal ameaça à validade do estudo?

A RTX 5060 Ti ajusta clocks em função de temperatura e potência. Sem fixar application clocks ou sem warm-up adequado, repetições iniciais rodam em clocks diferentes das finais, inflando o desvio padrão e enviesando a média.

17. Qual cuidado metodológico principal será adotado?

Antes de cada configuração de N, executar 5 iterações descartadas para aquecer a GPU e estabilizar clocks. Registrar a temperatura da GPU antes e depois de cada bloco de 30 repetições. Se a temperatura variar mais que 5 °C dentro de um bloco, sinalizar como observação no dado bruto.
